

An allocator-aware variant type

Abstract

Library types that can potentially hold *allocator-aware* (AA) objects should, themselves, be allocator-aware. A PMR container, for example, depends on AA types following a known AA protocol so that it (the container) can uniformly manage its memory. Even types that don't manage their own memory, such as `tuple`, follow the AA rules when they hold one more more AA elements. (A special case is `pair`, which is not actually AA but effectively follows the rules through special handling in *uses-allocator construction*.)

The current definition of `std::variant` does not follow the rules for an AA type, even when holding an AA alternative. This limitation makes `std::variant` unsuitable for storage in an AA container when memory allocation customization is needed.

In this paper, we propose a new, allocator-aware `std::basic_variant` usable as a container element type of allocator aware containers, and in any other context where allocator propagation and allocator preservation is expected. This new type would not replace the current `std::variant` as the desired behaviour is not compatible with how `std::variant` handles allocator aware types at the moment. We also propose an `std::pmr::variant` type which is a specialisation of `std::basic_variant` for `std::pmr::polymorphic_allocator<>`.

This is a complete proposal with formal wording.

Motivation

General Motivation for allocator-aware types

Note: The text below is borrowed nearly verbatim from [P3002](#), which proposes a general policy for when types should use allocators

Memory management is a major part of building software. Numerous facilities in the C++ Standard library exist to give the programmer maximum control over how their program uses memory:

- `std::unique_ptr` and `std::shared_ptr` are parameterized with *deleter* objects that control, among other things, how memory resources are reclaimed.
- `std::vector` is preferred over other containers in many cases because its use of contiguous memory provides optimal cache locality and minimizes allocate/deallocate operations. Indeed, the LEWG has spent a lot of time on `flat_set` ([P1222R0](#)) and `flat_map` ([P0429R3](#)), whose underlying structure defaults to `vector` for this reason.
- Operators `new` and `delete` are replaceable, giving programmers global control over how memory is allocated.
- The C++ object model makes a clear distinction between an object's memory footprint and its lifetime.
- Language constructs such as `void*` and `reinterpret_cast` provide fine-grained access to objects' underlying memory.
- Standard containers and strings are parameterized with allocators, providing object-level control over memory allocation and element construction.

This fine-grained control over memory that C++ gives the programmer is a large part of why C++ is applicable to so many domains — from embedded systems with limited memory budgets to games, high-frequency trading, and scientific simulations that require cache locality, thread affinity, and other memory-related performance optimizations.

An in-depth description of the value proposition for allocator-aware software can be found in [P2035R0](#). Standard containers are the most ubiquitous examples of *allocator-aware* types. Their `allocator_type` and `get_allocator` members and allocator-parameterized constructors allow them to be used like Lego® parts that can be combined and nested as necessary while retaining full programmer control over how the whole assembly allocates memory. For *scoped allocators* — those that apply not only to the top-level container, but also to its elements — having each element of a container support a predictable allocator-aware interface is crucial to giving the programmer the ability to allocate all memory from a single memory resource, such as an arena or pool. Note that the allocator is a *configuration* parameter of an object and does not contribute to its value.

In short, the principles underlying this policy proposal are:

1. [The Standard Library should be general and flexible](#). To the extent possible, the user of a library class should have control over how memory is allocated.
2. [The Standard Library should be consistent](#). The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.
3. [The parts of the Standard Library should work together](#). If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.
4. [The Standard Library should encapsulate complexity](#). Fully general application of allocators is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

Motivation for an Allocator-aware variant

Although each alternative in a `std::variant` can be initialized with any set of valid constructor arguments, including allocator arguments, the fact that the `variant` itself is not allocator-aware prevents it from working consistently with other parts of the standard library, specifically those parts that depend on *uses-allocator construction* (section [allocator.uses.construction]) in the standard). For example:

```
pmr::polymorphic_allocator<> alloc{ ... };
using V = variant<pmr::string, int>;
V v = make_obj_using_allocator<V>(alloc,
                                  in_place_type<pmr::string>, "hello");
assert(get<0>(v).get_allocator() == alloc); // FAILS
```

Even though an allocator is supplied, it is not used to construct the `pmr::string` within the resulting `variant` object because `variant` does not have the necessary hooks for `make_obj_using_allocator` to recognize it as being allocator-aware. Note that, although this example and the ones that follow use `pmr::polymorphic_allocator`, the same issues would apply to any scoped allocator.

Uses-allocator construction is rarely used directly in user code. Instead, it is used within the implementation of standard containers and scoped allocators to ensure that the allocator used to construct the container is also used to construct its elements. Continuing the example above, consider what happens if a `variant` is stored in a `pmr::vector`, compared to storing a truly allocator-aware type (`pmr::string`):

```
pmr::vector<pmr::string> vs(alloc);
pmr::vector<V> vv(alloc);

vs.emplace_back("hello");
vv.emplace_back("hello");

assert(vs.back().get_allocator() == alloc); // OK
assert(get<0>(vv.back()).get_allocator() == alloc); // FAILS
```

An important invariant when using a scoped allocator such as `pmr::polymorphic_allocator` is that the same allocator is used throughout an object hierarchy. It is impossible to ensure that this invariant is preserved when using `std::variant`, even if the element is originally inserted with the correct allocator, because `variant` does not remember the allocator used to construct it and cannot therefore re-instate the allocator when changing the active alternative:

```
vv.emplace_back(in_place_type<pmr::string>, "hello", alloc);
assert(get<0>(vv.back()).get_allocator() == alloc); // OK

vv.back() = 5; // Change alternative to `int`
vv.back() = "goodbye"; // Change alternative back to `pmr::string`
assert(get<0>(vv.back()).get_allocator() == alloc); // FAILS
```

Finally, when using assignment, the value stored in the `variant` is set sometimes by construction and other times by assignment. Depending on the allocator type's propagation traits, it is difficult to reason about the resulting alternative's allocator:

```
V v1{ 5 }; // int does not use an allocator
V v2{ "hello" }; // string uses a default-constructed allocator
v1 = pmr::string("goodbye", alloc); // Constructs the string
v2 = pmr::string("goodbye", alloc); // Assigns to the string
assert(get<0>(v1).get_allocator() == alloc); // OK, set by move construction
assert(get<0>(v2).get_allocator() == alloc); // ERROR, set by assignment
```

Summary of the proposed feature

This paper proposes an allocator-aware `variant`. Unfortunately, it would be complicated to add an allocator to the current `std::variant` without causing API and ABI compatibility issues and/or imposing long compile times on code that does not benefit from the change. For this reason, we are

proposing a new class template, `basic_variant`, which works like `variant`, but adds allocator support.

The key attributes of `basic_variant` that make it different from `variant` are:

- It has an `Allocator` template parameter.
- It has a public `allocator_type` member to advertise that it is allocator-aware.
- There are allocator-enabled versions of every constructor.
- The supplied allocator is forwarded to the constructor of the active alternative, if that alternative uses a compatible allocator type. The allocator is retained so that it can be supplied to the contained object whenever the active alternative changes. Allocator traits, including propagation traits, are respected.
- It has an alias, `pmr::variant`, for which `Allocator` is hard-coded to `pmr::polymorphic_allocator<>`.

Design decisions:

Basic_variant supports non-scoped propagating allocators

There are two ways of viewing `basic_variant` from allocator propagation perspective :

#1 `basic_varian` is like an `std::tuple`, i.e. it only accepts the allocator at construction so it can forward it to the `value_type` object. One can use a non-scoped propagation allocator, and when using a scoped propagation allocator `basic_varian` will not "consume" an allocator level. A variant object is in a way like a tuple object as it does not use the allocator itself, it only passes it into the `value_type` object.

#2 `basic_variant` is like an `std::vector`, i.e. it is a container of one or zero elements, and one should use a scoped propagating allocator if one wants the `value_type` object to use the allocator. In this approach `basic_variant` will "consume" an allocator level. Using non-scoped propagating allocators makes little sense in this scenario.

The proposal implements #1 as `basic_variant` itself does not allocate any memory so it makes little sense for it to consume an allocator level. The allocator in `basic_variant` is simply stored so it can be used by any alternative that requires it.

Allocator overhead

Allocator overhead can be removed in two cases

- if all alternatives are not allocator aware
- is `allocator_traits<Allocator>::is_always_equal::value == true`

In the former case, allocator is simply not needed. In the latter case, there is no need to store or pass in the allocator as the default allocator will do. The paper doesn't standardise such optimisation, it is left as a QOI. Note that the above cases reduce to `std::variant` behaviour.

Default constructor considerations

`Uses_allocator` construction is always used in case the default allocator construction produces different allocators each time. Possible optimisation opportunity for allocators which have `is_always_equal` set to true, but no benefit in triviality or `constexpr` - `uses_allocator` construction will be `constexpr` if it can be `constexpr` either way. Possible additional consideration is for `basic_variant` where no types are allocator aware - such a `basic_variant` `constexpr` considerations would only depend on the `constexpr` construction of the 0th alternative.

No conditional triviality of the default constructor is possible - the internal state needs to be initialised.

Because default constructor has to explicitly specify the allocator, unless `is_always_equal==true`, the default constructor is `noexcept` if 0th alternative is not AA or `is_always_equal==true`, and default construction of 0th alternative is `noexcept`.

Copy constructor considerations

Copy construction does not do `uses_allocator` construction - the type is expected to behave according to the traits and get the right allocator. This allows for triviality if allocator is always equal and both the allocator and the constructed type can be trivially copy constructed. If allocator is not always equal, we need to check the traits to get the right allocator and that initialisation can not be trivial. Possible additional consideration is for `basic_variant` where no types are allocator aware - such a `basic_variant` does not need to store the allocator and triviality only depends on the triviality of the alternatives. This proposal does not consider such optimisation.

Move constructor considerations

Move construction does not do `uses_allocator` construction - the type is expected to behave according to the traits and get the right allocator. This allows for triviality if allocator and the constructed type can be trivially move constructed. Possible additional consideration is for `basic_variant` where no types

are allocator aware - such a `basic_variant` does not need to store the allocator and triviality only depends on the triviality of the alternatives. This proposal does not consider such optimisation.

Value constructor considerations

Value construction must do `uses_allocator` construction if allocators aren't always equal- the plain copy/move construction may use the wrong allocator.

The paper does not propose exception specification on this value constructor and considers it QOI

Exception specification considerations :

- for non AA type, the exception specification is equivalent to `is_nothrow_constructible_v<Tj, T>`
- for AA type, to get a possibly no throw construction, we need to delegate to plain copy/move which can possibly reuse the allocation. The delegation can only happen if allocators are always equal. If this constructor checked for allocators being always equal before deciding whether to do `uses_allocator` construction or not, it would make sense for all inplace constructors and allocator extended constructors to do the same. This paper does not propose such considerations.

Copy assignment operator considerations

First the allocator is possibly modified based on the traits, then the assignment/construction is done as normal. In addition to `std::variant`'s triviality requirements, triviality can only happen if allocator is always equal and it is a trivially copyable type. The condition for the copy construction vs variant move construction is as follows :

- if a copy construction can't throw do copy construction
- if move construction can throw do copy construction as the move construction doesn't bring any benefits
- otherwise, do move construction.

With the introduction of allocators, we now have to consider when we can do plain copy construction and when allocator extended construction is needed. If type is non AA, we can do plain copy construction. If the type is AA and allocator is always equal, we can also do plain copy construction. Note that, for allocator is always equal case, plain copy construction and allocator copy construction will effectively result in the same allocator. We do not consider the possibility of doing non allocator extended copy construction for the case where the copy assignment allocator propagation matches the resulting allocator at copy construction as that case can't be checked at compile time.

In all other cases, copy construction must use allocator extended copy construction to guarantee that the right allocator is used. We assume such cases are always throwing.

Move assignment from a temporary `basic_variant` must construct the temporary object with `uses_allocator` construction to ensure correct allocator is used. A possible optimisation is for the case when allocators are always equal or if `POCMA=true` - such temporary could be constructed with a non allocator extended constructor. This paper does not propose such optimisation.

Move assignment operator

First the allocator is possibly modified based on the traits, then the assignment/construction is done as normal. In addition to usual triviality requirements, triviality can only happen if the allocator is propagated for move assignment(`POCMA=true`) or if the allocator is always equal. Possible additional consideration is for `basic_variant` where no types are allocator aware - such a `basic_variant` does not need to ever allocate and triviality only depends on the triviality of the alternative operations. This proposal does not consider such optimisation.

The exception specification needs to consider assignment and construction operations. For AA types, move construction can be done directly if the allocator is always equal or if `POCMA=true`. Otherwise, the move construction needs to explicitly specify the allocator and we consider such move construction to always be possibly throwing.

Value assignment operator

Value construction must do `uses_allocator` construction if allocators aren't always equal- the plain copy/move construction may use the wrong allocator. Exception specification :

- for non AA type, the exception specification is equivalent to `is_nothrow_assignable_v<Tj, T> && is_nothrow_constructible_v<Tj, T>`
- for AA type, any allocation is potentially throwing. To get a possibly no throw construction, we need to delegate to plain move which can possibly reuse the allocation. The delegation can only happen if allocators are always equal.

Possible additional consideration is for `basic_variant` where no types are allocator aware - such a `basic_variant` does not need to ever allocate and exception specification depends only on the alternatives This proposal does not consider such optimisation.

Member swap

First the allocator is possibly modified based on the traits, then the swap/construction is done as normal. Exception specification : To get a possibly no throw construction, we need to delegate to plain move which can possibly reuse the allocation. The delegation can only happen if allocators are always equal or if `propagate_on_container_swap == true` (for such a type, the resulting allocator in move construction is the required allocator as move construction always propagates the allocator.) This means that swap can only be `noexcept` if

- `is_nothrow_move_constructible_v<Ti, T> && is_nothrow_constructible_v<Ti, T>` for all i, and
- allocator is always equal or `propagate_on_container_swap == true`

Note that swap has undefined behaviour if (allocator is always equal or propagate_on_container_swap == true) condition is not satisfied, so this condition is not reflected in the exception specification.

Proposed wording

Modify 22.6.2 Header <variant> synopsis [variant.syn]

```
// mostly freestanding
#include <compare>           // see [compare.syn]

namespace std {
    // [variant.variant], class template variant
    template<class... Types>
        class variant;

    // [variant.basic.variant], class template basic_variant
    template<class Allocator, class... Types>
        class basic_variant;

    // [variant.helper], variant helper classes
    template<class T> struct variant_size;           // not defined
    template<class T> struct variant_size<const T>;
    template<class T>
        constexpr size_t variant_size_v = variant_size<T>::value;

    template<class... Types>
        struct variant_size<variant<Types...>>;
    template<class... Types>
        struct variant_size<basic_variant<Allocator, Types...>>;

    template<size_t I, class T> struct variant_alternative;    // not defined
    template<size_t I, class T> struct variant_alternative<I, const T>;
    template<size_t I, class T>
        using variant_alternative_t = typename variant_alternative<I, T>::type;

    template<size_t I, class... Types>
        struct variant_alternative<I, variant<Types...>>;
    template<size_t I, class Allocator, class... Types>
        struct variant_alternative<I, basic_variant<Allocator, Types...>>;

    inline constexpr size_t variant_npos = -1;

    // [variant.get], value access
    template<class T, class... Types>
        constexpr bool holds_alternative(const variant<Types...>&) noexcept;
    template<class T, class Allocator, class... Types>
        constexpr bool holds_alternative(const basic_variant<Allocator, Types...>&) noexcept;

    template<size_t I, class... Types>
        constexpr variant_alternative_t<I, variant<Types...>>&
            get(variant<Types...>&);           // freestanding-deleted
    template<size_t I, class... Types>
        constexpr variant_alternative_t<I, variant<Types...>>&&
            get(variant<Types...>&&);           // freestanding-deleted
    template<size_t I, class... Types>
        constexpr const variant_alternative_t<I, variant<Types...>>&
            get(const variant<Types...>&);           // freestanding-deleted
    template<size_t I, class... Types>
        constexpr const variant_alternative_t<I, variant<Types...>>&&
            get(const variant<Types...>&&);           // freestanding-deleted

    template<size_t I, class Allocator, class... Types>
```

```
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&
    get(basic_variant<Allocator, Types...>&);           // freestanding-deleted
template<size_t I, class Allocator, class... Types>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&&
    get(basic_variant<Allocator, Types...>&&);           // freestanding-deleted
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>&
    get(const basic_variant<Allocator, Types...>&);       // freestanding-deleted
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>&&
    get(const basic_variant<Allocator, Types...>&&);       // freestanding-deleted
```

```
template<class T, class... Types>
constexpr T& get(variant<Types...>&);                   // freestanding-deleted
template<class T, class... Types>
constexpr T&& get(variant<Types...>&&);                 // freestanding-deleted
template<class T, class... Types>
constexpr const T& get(const variant<Types...>&);       // freestanding-deleted
template<class T, class... Types>
constexpr const T&& get(const variant<Types...>&&);     // freestanding-deleted
```

```
template<class T, class Allocator, class... Types>
constexpr T& get(basic_variant<Allocator, Types...>&); // freestanding-deleted
template<class T, class Allocator, class... Types>
constexpr T&& get(basic_variant<Allocator, Types...>&&); // freestanding-deleted
template<class T, class Allocator, class... Types>
constexpr const T& get(const basic_variant<Allocator, Types...>&); // freestanding-deleted
template<class T, class Allocator, class... Types>
constexpr const T&& get(const basic_variant<Allocator, Types...>&&); // freestanding-deleted
```

```
template<size_t I, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>>
    get_if(variant<Types...>*) noexcept;
template<size_t I, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>>
    get_if(const variant<Types...>*) noexcept;
```

```
template<size_t I, class Allocator, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, basic_variant<Allocator, Types...>>>
    get_if(basic_variant<Allocator, Types...>*) noexcept;
template<size_t I, class Allocator, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, basic_variant<Allocator, Types...>>>
    get_if(const basic_variant<Allocator, Types...>*) noexcept;
```

```
template<class T, class... Types>
constexpr add_pointer_t<T>
    get_if(variant<Types...>*) noexcept;
template<class T, class... Types>
constexpr add_pointer_t<const T>
    get_if(const variant<Types...>*) noexcept;
```

```
template<class T, class Allocator, class... Types>
constexpr add_pointer_t<T>
    get_if(basic_variant<Allocator, Types...>*) noexcept;
template<class T, class Allocator, class... Types>
constexpr add_pointer_t<const T>
    get_if(const basic_variant<Allocator, Types...>*) noexcept;
```

// [variant.relops], relational operators

```
template<class... Types>
constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
constexpr bool operator!=(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
constexpr bool operator<(const variant<Types...>&, const variant<Types...>&);
```

```

template<class... Types>
constexpr bool operator>(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
constexpr bool operator>=(const variant<Types...>&, const variant<Types...>&);
template<class... Types> requires (three_way_comparable<Types> && ...)
constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
operator<=>(const variant<Types...>&, const variant<Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator==(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator!=(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator<(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator>(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator<=(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<class AllocatorLhs, AllocatorRhs, class... Types>
constexpr bool operator>=(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);
template<AllocatorLhs, AllocatorRhs, class... Types> requires (three_way_comparable<Types> && ...)
constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
operator<=>(const basic_variant<AllocatorLhs, Types...>&, const basic_variant<AllocatorRhs, Types...>&);

```

// [variant.visit], visitation

```

template<class Visitor, class... Variants>
constexpr see below visit(Visitor&&, Variants&&...);
template<class R, class Visitor, class... Variants>
constexpr R visit(Visitor&&, Variants&&...);

```

// [variant.monostate], class monostate

```

struct monostate;

```

// [variant.monostate.relops], monostate relational operators

```

constexpr bool operator==(monostate, monostate) noexcept;
constexpr strong_ordering operator<=>(monostate, monostate) noexcept;

```

// [variant.specalg], specialized algorithms

```

template<class... Types>
constexpr void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);

```

```

template<class Allocator, class... Types>
constexpr void swap(basic_variant<Allocator, Types...>&, basic_variant<Allocator, Types...>&) noexcept(see below);

```

// [variant.bad.access], class bad_variant_access

```

class bad_variant_access;

```

// [variant.hash], hash support

```

template<class T> struct hash;
template<class... Types> struct hash<variant<Types...>>;
template<class Allocator, class... Types> struct hash<basic_variant<Allocator, Types...>>;
template<> struct hash<monostate>;

```

```

namespace pmr {
    template<class... Types>
        using variant = basic_variant<polymorphic_allocator<>, Types...>;
}

```

Insert new section after [22.6.3](#) Class template variant [\[variant.variant\]](#)

22.6.x Class template basic_variant [variant.basic.variant]

X.x.x. 1 General [variant.basic.variant.general]

```

namespace std {

```

```
template<class Allocator, class... Types>
class basic_variant {
public:
```

```
    using allocator_type = Allocator;
```

```
// [variant.basic.ctor], constructors
constexpr basic_variant() noexcept(see below);
constexpr basic_variant(const basic_variant&);
constexpr basic_variant(basic_variant&&) noexcept(see below);
```

```
template<class T>
    constexpr basic_variant(T&&);
template<class AllocatorU>
    constexpr basic_variant(const basic_variant<U, Types...>&);
template<class AllocatorU>
    constexpr basic_variant(basic_variant<U, Types...>&&) noexcept(see below);
    constexpr basic_variant(const variant<Types...>&);
    constexpr basic_variant(variant<Types...>&&);
```

```
template<class T, class... Args>
    constexpr explicit basic_variant(in_place_type_t<T>, Args&&...);
template<class T, class U, class... Args>
    constexpr explicit basic_variant(in_place_type_t<T>, initializer_list<U>, Args&&...);
template<size_t I, class... Args>
    constexpr explicit basic_variant(in_place_index_t<I>, Args&&...);
template<size_t I, class U, class... Args>
    constexpr explicit basic_variant(in_place_index_t<I>, initializer_list<U>, Args&&...);
```

```
// allocator-extended constructors
constexpr basic_variant(allocator_arg_t, const allocator_type a);
constexpr basic_variant(allocator_arg_t, const allocator_type a, const basic_variant&);
constexpr basic_variant(allocator_arg_t, const allocator_type a, basic_variant&&)
```

```
template<class T>
    constexpr basic_variant(allocator_arg_t, const allocator_type a, T&&);
template<class AllocatorU>
    constexpr basic_variant(allocator_arg_t, const allocator_type a, const basic_variant<U, Types...>&);
template<class AllocatorU>
    constexpr basic_variant(allocator_arg_t, const allocator_type a, basic_variant<U, Types...>&&);
    constexpr basic_variant(allocator_arg_t, const allocator_type a, const variant<Types...>&);
    constexpr basic_variant(allocator_arg_t, const allocator_type a, variant<Types...>&&);
```

```
template<class T, class... Args>
    constexpr explicit basic_variant(allocator_arg_t, const allocator_type a, in_place_type_t<T>, Args&&...);
template<class T, class U, class... Args>
    constexpr explicit basic_variant(allocator_arg_t, const allocator_type a, in_place_type_t<T>, initializer_list<U>, Args&&...);
```

```
template<size_t I, class... Args>
    constexpr explicit basic_variant(allocator_arg_t, const allocator_type a, in_place_index_t<I>, Args&&...);
template<size_t I, class U, class... Args>
    constexpr explicit basic_variant(allocator_arg_t, const allocator_type a, in_place_index_t<I>, initializer_list<U>, Args&&...);
```

```
// [variant.basic.dtor], destructor
constexpr ~basic_variant();
```

```
// [variant.basic.assign] assignment
constexpr basic_variant& operator=(const basic_variant&);
constexpr basic_variant& operator=(basic_variant&&) noexcept(see below);
template<class AllocatorU>
    constexpr basic_variant& operator=(const basic_variant<U, Types...>&);
template<class AllocatorU>
    constexpr basic_variant& operator=(basic_variant<U, Types...>&&) noexcept(see below);
    constexpr basic_variant& operator=(const variant<Types...>&);
    constexpr basic_variant& operator=(variant<Types...>&&) noexcept(see below);
```



```
template<class T> constexpr basic_variant& operator=(T&&) noexcept(see below);
```

```
// [variant.basic.mod], modifiers
template<class T, class... Args>
constexpr T& emplace(Args&&...);
template<class T, class U, class... Args>
constexpr T& emplace(initializer_list<U>, Args&&...);
template<size_t I, class... Args>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>& emplace(Args&&...);
template<size_t I, class U, class... Args>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&
emplace(initializer_list<U>, Args&&...);
```

```
// [variant.basic.status], value status
constexpr bool valueless_by_exception() const noexcept;
constexpr size_t index() const noexcept;
```

```
// [variant.basic.al], allocator
allocator_type get_allocator() const;
```

```
// [variant.basic.swap], swap
constexpr void swap(basic_variant&) noexcept(see below);
```

```
// [variant.basic.visit] visitation
template<class Self, class Visitor>
constexpr decltype(auto) visit(this Self&&, Visitor&&);
template<class R, class Self, class Visitor>
constexpr R visit(this Self&&, Visitor&&);
private:
    allocator_type alloc;    // exposition only
};
}
```

Any instance of `basic_variant` at any given time either holds a value of one of its alternative types or holds no value. When an instance of `basic_variant` holds a value of alternative type `T`, it means that a value of type `T`, referred to as the `basic_variant` object's [contained value](#), is allocated within the storage of the `basic_variant` object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the contained value.

All types in `Types` shall meet the *Cpp17Destructible* requirements (Table 35)

A program that instantiates the definition of `basic_variant` with less than 2 template arguments is ill-formed.

Template argument `Allocator` shall satisfy the *Cpp17Allocator* requirements (16.4.4.6.1). A an instance of `Allocator` is maintained by the `basic_variant` object during the lifetime of the object or until the allocator is replaced. The allocator instance is set at object creation time and may be replaced only via assignment or swap as specified below. The allocator instance is used to pass to uses-allocator construction of an alternative as specified below.

If a program declares an explicit or partial specialization of `basic_variant`, the program is ill-formed, no diagnostic required

X.x.x.2 Constructors [variant.basic.ctor]

In the descriptions that follow, let i be in the range $[0, \text{sizeof}...(Types))$, and T_i be the i^{th} type in `Types`.

```
constexpr basic_variant() noexcept(see below);
```

Constraints: `is_default_constructible_v<T0>` is true.

Effects: `alloc` is default-initialized. If `allocator_traits<Allocator>::is_always_equal::value` is false, an alternative of type `T0` is constructed by uses-allocator construction with allocator `alloc` and no constructor arguments. Otherwise, `T0` is value-initialized.

Postconditions: `valueless_by_exception()` is false and `index()` is 0.

Throws: Any exception thrown by the value-initialization of `T0`.

Remarks: This function is constexpr if and only if the selected constructor of the alternative type `T0` and the allocator `alloc` is constexpr-suitable ([dcl.constexpr]). The exception specification is equivalent to `is_nothrow_default_constructible_v<T0> && (allocator_traits<Allocator>::is_always_equal::value || !uses_allocator_v<remove_cv_t<T0>>)`. [Note : See also class `monostate` — end note]

```
constexpr basic_variant(const basic_variant& w);
```

Effects: If `allocator_traits<Allocator>::is_always_equal::value` is true, `alloc` is default-initialized. Otherwise, `alloc` is direct-initialized with an `allocator_traits<Allocator>::select_on_container_copy_construction(w.get_allocator())`. If `w` holds a value, initializes the variant to hold the same alternative as `w` and direct-initializes the contained value with `GET<j>(w)`, where `j` is `w.index()`. Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by direct-initializing any T_i for all i .

Remarks: This constructor is defined as deleted unless `is_copy_constructible_v< $T_i$ >` is true for all i . This constructor is trivial if

- `is_trivially_copy_constructible_v< $T_i$ >` is true for all i ,
- `allocator_traits<Allocator>::is_always_equal::value` is true, and
- `is_trivially_copy_constructible_v<Allocator>` is true

```
constexpr basic_variant(basic_variant && w) noexcept(see below);
```

Constraints: `is_move_constructible_v< $T_i$ >` is true for all i .

Effects: `alloc` is direct-initialized with `w.get_allocator()`. If `w` holds a value, initializes the variant to hold the same alternative as `w` and direct-initializes the contained value with `GET<j>(std :: move(w))`, where `j` is `w.index()`. Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by move-constructing any T_i for all i .

Remarks: The exception specification is equivalent to the logical and of `is_nothrow_move_constructible_v< $T_i$ >` for all i . This constructor is trivial if :

- `is_trivially_move_constructible_v< $T_i$ >` is true for all i , and
- `is_trivially_move_constructible_v<Allocator>` is true.

```
template<class T> constexpr basic_variant(T&& t) ;
```

Let T_j be a type that is determined as follows: build an imaginary function $FUN(T_i)$ for each alternative type T_i for which $T_i \times [] = \{\text{std} :: \text{forward}<T>(t)\}$; is well-formed for some invented variable x_i . The overload $FUN(T_j)$ selected by overload resolution for the expression $FUN(\text{std} :: \text{forward}<T>(t))$ defines the alternative T_j which is the type of the contained value after construction.

Constraints:

- `sizeof...(Types)` is nonzero,
- `is_same_v<remove_cvref_t<T>, variant>` is false,
- `remove_cvref_t<T>` is neither a specialization of `in_place_type_t` nor a specialization of `in_place_index_t`,
- `is_constructible_v< $T_j$ , T>` is true, and the expression $FUN(\text{std} :: \text{forward}<T>(t))$ (with FUN being the above-mentioned set of imaginary functions) is well-formed.

[Note :

```
basic_variant<string, string> v("abc");
```

- o is ill-formed, as both alternative types have an equally viable constructor for the argument. — end note]

Effects: `alloc` is default-initialized. If `allocator_traits<Allocator>::is_always_equal::value` is false, an alternative of type T_j is constructed by uses-allocator construction with `allocator alloc` and `std::forward<T>(t)`. Otherwise, an alternative of type T_j is direct-non-list-initialized with `std::forward<T>(t)`.

Postconditions: `holds_alternative< $T_j$ >(*this)` is true.

Throws: Any exception thrown by the initialization of the selected alternative T_j .

Remarks: If `Allocator`'s default constructor and T_j 's selected constructor are `constexpr` constructors, this constructor is a `constexpr` constructor.

```
template<class AllocatorU> constexpr explicit basic_variant(const basic_variant<AllocatorU, Types...>& w);
```

Constraints: `is_copy_constructible_v< $T_i$ >` is true for all i .

Effects: If `allocator_traits<Allocator>::is_always_equal::value` is true, `alloc` is default-initialized. Otherwise, `alloc` is direct-initialized with an `allocator_traits<Allocator>::select_on_container_copy_construction(w.get_allocator())`. If `w` holds a value, initializes the variant to hold the same alternative as `w` and direct-initializes the contained value with `GET<j>(w)`, where `j` is `w.index()`. Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by direct-initializing any T_i for all i .

Remarks: If `is_convertible_v<const AllocatorU&, Allocator>` is false, this constructor is defined as deleted. If `Allocator`'s default constructor and T_i 's selected constructor are `constexpr` constructors, this constructor is a `constexpr` constructor.

```
template<class AllocatorU> constexpr explicit basic_variant(basic_variant<AllocatorU, Types...>&& w) noexcept(see below);
```

Constraints: `is_move_constructible_v< $T_i$ >` is true for all i .

Effects: `alloc` is direct-initialized with `w.get_allocator()`. If `w` holds a value, initializes the variant to hold the same alternative as `w` and direct-initializes the contained value with `GET<j>(std :: move(w))`, where `j` is `w.index()`. Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by move-constructing any T_i for all i .

Remarks: The exception specification is equivalent to the logical and of `is_nothrow_move_constructible_v< $T_i$ >` for all i . If `is_convertible_v<const AllocatorU&, Allocator>` is false, this constructor is defined as deleted. If `Allocator`'s default constructor and T_i 's

selected constructor are constexpr constructors, this constructor is a constexpr constructor.

constexpr explicit basic_variant(const variant<Types...>& w);

Constraints: is_copy_constructible_v<T_i> is true for all *i*.

Effects: alloc is default-initialized. If w holds a value, initializes the variant to hold the same alternative as w and constructs the contained value by uses-allocator construction with allocator alloc and GET<j>(w), where j is w.index(). Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by move-constructing any T_i for all.

Remarks: If Allocator's default constructor and T_i's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

constexpr explicit basic_variant(variant<Types...>&& w);

Constraints: is_move_constructible_v<T_i> is true for all *i*.

Effects: alloc is default-initialized. If w holds a value, initializes the variant to hold the same alternative as w and constructs the contained value by uses-allocator construction with allocator alloc and GET<j>(std::move(w)), where j is w.index(). Otherwise, initializes the variant to not hold a value.

Throws: Any exception thrown by move-constructing any T_i for all.

Remarks: If Allocator's default constructor and T_i's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<class T, class U, class... Args>

constexpr explicit basic_variant(in_place_type_t<T>, initializer_list<U> il, Args&&... args);

Constraints:

- There is exactly one occurrence of T in Types... and
- is_constructible_v<T, initializer_list<U>&, Args...> is true.

Effects: alloc is default-initialized. An alternative of type T is constructed by uses-allocator construction with allocator alloc and il, std::forward<Args>(args)...

Postconditions: holds_alternative<T>(*this) is true.

Throws: Any exception thrown by calling the selected constructor of T.

Remarks: If Allocator's default constructor and T's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<size_t I, class... Args> constexpr explicit basic_variant(in_place_index_t<I>, Args&&... args);

Constraints:

- I is less than sizeof...(Types) and
- is_constructible_v<T_I, Args...> is true.

Effects: alloc is default-initialized. An alternative of type T_I is constructed by uses-allocator construction with allocator alloc and std::forward<Args>(args)...

Postconditions: index() is I.

Throws: Any exception thrown by calling the selected constructor of T_I.

Remarks: If Allocator's default constructor and T_I's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

template<size_t I, class U, class... Args>

constexpr explicit basic_variant(in_place_index_t<I>, initializer_list<U> il, Args&&... args);

Constraints:

- I is less than sizeof...(Types) and
- is_constructible_v<T_I, initializer_list<U>&, Args...> is true.

Effects: alloc is default-initialized. An alternative of type T_I is constructed by uses-allocator construction with allocator alloc and il, std::forward<Args>(args)...

Postconditions: index() is I.

Throws: Any exception thrown by calling the selected constructor of T_I.

Remarks: If Allocator's default constructor and T_I's selected constructor are constexpr constructors, this constructor is a constexpr constructor.

constexpr basic_variant(allocator_arg_t, const Allocator& a);

constexpr basic_variant(allocator_arg_t, const Allocator& a, const basic_variant& w);

constexpr basic_variant(allocator_arg_t, const Allocator& a, basic_variant&& w);

template<class T> constexpr basic_variant(allocator_arg_t, const Allocator& a, T&& t);

template<class AllocatorU> constexpr basic_variant(allocator_arg_t, const allocator_type a, const basic_variant<U, Types...>&);

template<class AllocatorU> constexpr basic_variant(allocator_arg_t, const allocator_type a, basic_variant<U, Types...>&&);

constexpr basic_variant(allocator_arg_t, const allocator_type a, const variant<Types...>&);

constexpr basic_variant(allocator_arg_t, const allocator_type a, variant<Types...>&&);

```
template<class T, class... Args> constexpr explicit basic_variant(allocator_arg_t, const Allocator& a, in_place_type_t<T>, Args&&... args);
template<class T, class U, class... Args>
constexpr explicit basic_variant(allocator_arg_t, const Allocator& a, in_place_type_t<T>, initializer_list<U> il, Args&&... args);
template<size_t I, class... Args> constexpr explicit basic_variant(allocator_arg_t, const Allocator& a, in_place_index_t<I>, Args&&... args);
template<size_t I, class U, class... Args>
constexpr explicit basic_variant(allocator_arg_t, const Allocator& a, in_place_index_t<I>, initializer_list<U> il, Args&&... args);
Effects: Behaves the same as non allocator extended version of the constructor except it initializes alloc with the specified allocator before initializing the alternative, if any, by uses-allocator construction.
```

[Question to LWG : do we need to specify how triviality differs for these constructors?]

x.x.x.x Destructor [variant.basic.dtor]

```
constexpr ~basic_variant();
```

Effects: If `valueless_by_exception()` is false, destroys the currently contained value.
Remarks: If `is_trivially_destructible_v<Ti>` is true for all T_i , then this destructor is trivial.

x.x.x.x Assignment [variant.basic.assign]

```
constexpr basic_variant& operator=(const basic_variant& rhs);
    Let  $j$  be rhs.index().
    Effects:
        - If allocator_traits<Allocator>::propagate_on_container_copy_assignment::value is true, sets alloc to rhs.alloc
        - If neither *this nor rhs holds a value, there is no additional effect.
        - Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value
        - Otherwise, if index() == j, assigns the value contained in rhs to the value contained in *this,
        - Otherwise, if
            and
            - uses_allocator_v<remove_cv_t<Tj>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true,
            - either is_nothrow_copy_constructible_v<Tj> is true or is_nothrow_move_constructible_v<Tj> is false,
              destroys the current alternative, if any, and direct-non-list-initializes the contained value of type  $T_j$  with GET<j>(rhs).
            - Otherwise, equivalent to operator=(basic_variant(allocator_arg, alloc, rhs)).
    Postconditions: index() == rhs.index()
    Returns: *this.
    Remarks: This operator is defined as deleted unless is_copy_constructible_v<Ti> && is_copy_assignable_v<Ti> is true for all  $i$ . The assignment operator is trivial if :
        - is_trivially_copy_constructible_v<Ti> && is_trivially_copy_assignable_v<Ti> && is_trivially_destructible_v<Ti> is true for all  $i$ ,
        - is_trivially_copy_constructible_v<Allocator> && is_trivially_copy_assignable_v<Allocator> && is_trivially_destructible_v<Allocator> is true, and
        - allocator_traits<Allocator>::is_always_equal::value is true.
```

```
constexpr basic_variant& operator=(basic_variant&& rhs) noexcept(see below);
    Let  $j$  be rhs.index().
    Constraints: is_move_constructible_v<Ti> && is_move_assignable_v<Ti> is true for all  $i$ .
    Effects:
        - If allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, sets alloc to rhs.alloc
        - If neither *this nor rhs holds a value, there is no effect.
        - Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value.
        - Otherwise, if index() == j, assigns GET<j>(std::move(rhs)) to the value contained in *this.
        - Otherwise, if allocator_traits<Allocator>::is_always_equal::value is true or allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, destroys the current alternative, if any, and direct-non-list-initializes the contained value of type  $T_j$  with GET<j>(std::move(rhs)).
        - Otherwise equivalent to emplace<j>(GET<j>(std::move(rhs))).
    Returns: *this.
    Remarks: If
        - is_trivially_move_constructible_v<Ti> && is_trivially_move_assignable_v<Ti> && is_trivially_destructible_v<Ti> is true for all  $i$ , and
        - is_trivially_move_constructible_v<Allocator> && is_trivially_move_assignable_v<Allocator> && is_trivially_destructible_v<Allocator> is true
```

- allocator_traits<Allocator>::is_always_equal::value is true or allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true
 this assignment operator is trivial.
 The exception specification is equivalent to logical and operation of (allocator_traits<Allocator>::is_always_equal::value && allocator_traits<Allocator>::propagate_on_container_move_assignment::value) and is_nothrow_move_constructible_v<T_i> && is_nothrow_move_assignable_v<T_i> for all *i*.

If an exception is thrown during the call to T_j's move construction (with *j* being rhs.index()), the variant will hold no value.

If an exception is thrown during the call to T_j's move assignment, the state of the contained value is as defined by the exception safety guarantee of T_j's move assignment; index() will be *j*.

template<class T> constexpr basic_variant& operator=(T&& t) noexcept(see below);

Let T_j be a type that is determined as follows: build an imaginary function FUN(T_i) for each alternative type T_i for which T_i x[] = {std::forward<T>(t)}; is well-formed for some invented variable *x*. The overload FUN(T_j) selected by overload resolution for the expression FUN(std::forward<T>(t)) defines the alternative T_j which is the type of the contained value after assignment.

Constraints:

- is_same_v<remove_cvref_t<T>, variant> is false,
- is_assignable_v<T_j&, T> && is_constructible_v<T_j, T> is true, and
- the expression FUN(std::forward<T>(t)) (with FUN being the above-mentioned set of imaginary functions) is well-formed.

[Note :

variant<string, string> v;

▪ v = "abc";

is ill-formed, as both alternative types have an equally viable constructor for the argument. — end note]

Effects:

- If *this holds a T_j, assigns std::forward<T>(t) to the value contained in *this.

- Otherwise, if

- uses_allocator_v<remove_cvref_t<T_j>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true,

and

- either is_nothrow_copy_constructible_v<T_j> is true or is_nothrow_move_constructible_v<T_j> is false,

destroys the current alternative, if any, and direct-non-list-initializes the contained value of type T_j with std::forward<T>(t).

- Otherwise, equivalent to emplace<j>(T_j(std::forward<T>(t))).

Postconditions: holds_alternative<T_j>(*this) is true, with T_j selected by the imaginary function overload resolution described above.

Returns: *this.

Remarks: The exception specification is equivalent to: is_nothrow_assignable_v<T_j&, T> && is_nothrow_constructible_v<T_j, T> && (!uses_allocator_v<remove_cvref_t<T_j>, Allocator> || allocator_traits<Allocator>::is_always_equal::value)

If an exception is thrown during the assignment of std::forward<T>(t) to the value contained in *this, the state of the contained value and *t* are as defined by the exception safety guarantee of the assignment expression; valueless_by_exception() will be false.

If an exception is thrown during the initialization of the contained value, the variant object is permitted to not hold a value.

template<class AllocatorU>

constexpr basic_variant& operator=(const basic_variant<allocatorU, Types...>& rhs);

Let *j* be rhs.index().

Constraints: is_copy_constructible_v<T_i> && is_copy_assignable_v<T_i> is true for all *i*.

Effects:

- If allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, sets alloc to rhs.alloc

- If neither *this nor rhs holds a value, there is no additional effect.

- Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value

- Otherwise, if index() == *j*, assigns the value contained in rhs to the value contained in *this,

- Otherwise, if

- uses_allocator_v<remove_cv_t<T_j>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true,

and

- either is_nothrow_copy_constructible_v<T_j> is true or is_nothrow_move_constructible_v<T_j> is false,

destroys the current alternative, if any, and direct-non-list-initializes the contained value of type T_j with GET<j>(rhs)).

- Otherwise, equivalent to operator=(basic_variant(allocator_arg, alloc, rhs)).

Postconditions: index() == rhs.index()

Returns: *this.

Remarks: This operator is defined as deleted if is_convertible_v<const AllocatorU&, Allocator> is false.

template<class AllocatorU>

constexpr basic_variant& operator=(basic_variant<allocatorU, Types...> && rhs) noexcept(see below);

Let *j* be rhs.index().

Constraints: is_move_constructible_v<T_i> && is_move_assignable_v<T_i> is true for all *i*.

Effects:

- If allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, sets alloc to rhs.alloc

- If neither *this nor rhs holds a value, there is no effect.

- Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value.

- Otherwise, if index() == *j*, assigns GET<*j*>(std::move(rhs)) to the value contained in *this.

- Otherwise, if allocator_traits<Allocator>::is_always_equal::value is true or

allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, destroys the current alternative, if any, and direct-non-list-initializes the contained value of type T_j with GET<*j*>(std::move(rhs)).

- Otherwise equivalent to emplace<*j*>(GET<*j*>(std::move(rhs))).

Returns: *this.

Remarks:

The exception specification is equivalent to logical and operation of (allocator_traits<Allocator>::is_always_equal::value && allocator_traits<Allocator>::propagate_on_container_move_assignment::value) and

is_nothrow_move_constructible_v<T_i> && is_nothrow_move_assignable_v<T_i> for all *i*.

If an exception is thrown during the call to T_j's move construction (with *j* being rhs.index()), the variant will hold no value.

If an exception is thrown during the call to T_j's move assignment, the state of the contained value is as defined by the exception safety guarantee of T_j's move assignment; index() will be *j*.

This operator is defined as deleted if is_convertible_v<const AllocatorU&, Allocator> is false.

constexpr basic_variant& operator=(const variant<Types...>& rhs);

Let *j* be rhs.index().

Constraints: is_copy_constructible_v<T_i> && is_copy_assignable_v<T_i> is true for all *i*.

Effects:

- If neither *this nor rhs holds a value, there is no additional effect.

- Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value

- Otherwise, if index() == *j*, assigns the value contained in rhs to the value contained in *this.

- Otherwise, if

- uses_allocator_v<remove_cv_t<T_j>>, Allocator> is false or allocator_traits<Allocator>::is_always_equal::value is true,

and

- either is_nothrow_copy_constructible_v<T_j> is true or is_nothrow_move_constructible_v<T_j> is false,

- destroys the current alternative, if any, and direct-non-list-initializes the contained value of type T_j with GET<*j*>(rhs)).

- Otherwise, equivalent to operator=(basic_variant(allocator_arg, alloc, rhs)).

Postconditions: index() == rhs.index()

Returns: *this.

constexpr basic_variant& operator=(variant<Types...>&& rhs) noexcept(see below);

Let *j* be rhs.index().

Constraints: is_move_constructible_v<T_i> && is_move_assignable_v<T_i> is true for all *i*.

Effects:

- If neither *this nor rhs holds a value, there is no effect.

- Otherwise, if *this holds a value but rhs does not, destroys the value contained in *this and sets *this to not hold a value.

- Otherwise, if index() == *j*, assigns GET<*j*>(std::move(rhs)) to the value contained in *this.

- Otherwise,

allocator_traits<Allocator>::propagate_on_container_move_assignment::value is true, destroys the current alternative, if any, and direct-non-list-initializes the contained value of type T_j with GET<*j*>(std::move(rhs)).

- Otherwise equivalent to emplace<*j*>(GET<*j*>(std::move(rhs))).

Returns: *this.

Remarks:

The exception specification is equivalent to logical and operation of (allocator_traits<Allocator>::is_always_equal::value && allocator_traits<Allocator>::propagate_on_container_move_assignment::value) and

is_nothrow_move_constructible_v<T_i> && is_nothrow_move_assignable_v<T_i> for all *i*.

If an exception is thrown during the call to T_j's move construction (with *j* being rhs.index()), the variant will hold no value.

If an exception is thrown during the call to T_j's move assignment, the state of the contained value is as defined by the exception safety guarantee of T_j's move assignment; index() will be *j*.

Modifiers [variant.basic.mod]

template<class T, class... Args> constexpr T& emplace(Args&&... args);

Constraints: is_constructible_v<T, Args...> is true, and T occurs exactly once in Types.

Effects: Equivalent to:

```
return emplace<I>(std::forward<Args>(args)...);
where I is the zero-based index of T in Types.
```

```
template<class T, class U, class... Args>
constexpr T& emplace(initializer_list<U> il, Args&&... args);
Constraints: is_constructible_v<T, initializer_list<U>&, Args...> is true, and T occurs exactly once in Types.
Effects: Equivalent to:
    return emplace<I>(il, std::forward<Args>(args)...);
    where I is the zero-based index of T in Types.
```

```
template<size_t I, class... Args>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>& emplace(Args&&... args);
Mandates: I < sizeof...(Types).
Constraints: is_constructible_v<TI, Args...> is true.
Effects: Destroys the currently contained value if valueless_by_exception() is false. Constructs an alternative of type TI by uses-allocator construction with allocator alloc and std::forward<Args>(args)... .
Postconditions: index() is I.
Returns: A reference to the new contained value.
Throws: Any exception thrown during the initialization of the contained value.
Remarks: If an exception is thrown during the initialization of the contained value, the variant is permitted to not hold a value.
```

```
template<size_t I, class U, class... Args>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&
emplace(initializer_list<U> il, Args&&... args);
Mandates: I < sizeof...(Types).
Constraints: is_constructible_v<TI, initializer_list<U>&, Args...> is true.
Effects: Destroys the currently contained value if valueless_by_exception() is false. Constructs an alternative of type TI by uses-allocator construction with allocator alloc and il, std::forward<Args>(args)... .
Postconditions: index() is I.
Returns: A reference to the new contained value.
Throws: Any exception thrown during the initialization of the contained value.
Remarks: If an exception is thrown during the initialization of the contained value, the variant is permitted to not hold a value.
```

Value status [variant.basic.status]

```
constexpr bool valueless_by_exception() const noexcept;
Effects: Returns false if and only if the variant holds a value.
[Note 1: It is possible for a basic_variant to hold no value if an exception is thrown during a type-changing assignment or emplacement. The latter means that even a basic_variant<float, int> can become valueless_by_exception(), for instance by
    struct S { operator int() { throw 42; } };
    basic_variant<std::allocator<char>, float, int> v{12.f};
    v.emplace<1>(S());
— end note]
```

```
constexpr size_t index() const noexcept;
Effects: If valueless_by_exception() is true, returns variant_npos. Otherwise, returns the zero-based index of the alternative of the contained value.
```

Allocator [variant.basic.all]

```
allocator_type get_allocator() const;
Returns: A copy of the Allocator that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement.
```

Swap [variant.basic.swap]

```
constexpr void swap(basic_variant& rhs) noexcept(see below);
Mandates: is_move_constructible_v<Ti> is true for all i.
Preconditions: Each Ti meets the Cpp17Swappable requirements (\[swappable.requirements\])
Effects:
    - If allocator_traits<Allocator>::propagate_on_container_swap::value is true, then Allocator shall meet the Cpp17Swappable requirements and the allocators of *this and res shall also be exchanged by calling swap as described in \[swappable.requirements\]. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless *this.get_allocator() == rhs.get_allocator().
```


- If `valueless_by_exception()` && `rhs.valueless_by_exception()` no additional effect.
- Otherwise, if `index() == rhs.index()`, calls `swap(GET<i>(*this), GET<i>(rhs))` where *i* is `index()`.
- Otherwise, exchanges values of `rhs` and `*this`.

Throws: If `index() == rhs.index()`, any exception thrown by `swap(GET<i>(*this), GET<i>(rhs))` with *i* being `index()`. Otherwise, any exception thrown by the move constructor of `Ti` or `Tj` with *i* being `index()` and *j* being `rhs.index()`.

Remarks: If an exception is thrown during the call to function `swap(GET<i>(*this), GET<i>(rhs))`, the states of the contained values of `*this` and of `rhs` are determined by the exception safety guarantee of `swap` for lvalues of `Ti` with *i* being `index()`. If an exception is thrown during the exchange of the values of `*this` and `rhs`, the states of the values of `*this` and of `rhs` are determined by the exception safety guarantee of variant's move constructor. The exception specification is equivalent to the logical and of `is_nothrow_move_constructible_v<Ti>` && `is_nothrow_swappable_v<Ti>` for all *i*.

Add to 22.6.4 variant helper classes

```
template<class... Types>
struct variant_size<variant<Types...>> : integral_constant<size_t, sizeof...(Types)> { };
template<class Allocator, class... Types>
struct variant_size<basic_variant<Allocator, Types...>> : integral_constant<size_t, sizeof...(Types)> { };
```

...

```
variant_alternative<I, variant<Types...>>::type
variant_alternative<I, basic_variant<Allocator, Types...>>::type
Mandates: I < sizeof...(Types).
Type: The type TI.
```

Add to 22.6.5 Value access

```
template<class T, class... Types>
constexpr bool holds_alternative(const variant<Types...>& v) noexcept;
template<class T, class Allocator, class... Types>
constexpr bool holds_alternative(const basic_variant<Allocator, Types...>& v) noexcept;
Mandates: The type T occurs exactly once in Types.
Returns: true if index() is equal to the zero-based index of T in Types
```

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&
GET(variant<Types...>& v); // exposition only
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&&
GET(variant<Types...>&& v); // exposition only
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&
GET(const variant<Types...>& v); // exposition only
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&&
GET(const variant<Types...>&& v); // exposition only
template<size_t I, class Allocator, class... Types>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&
GET(basic_variant<Allocator, Types...>& v); // exposition only
template<size_t I, class Allocator, class... Types>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&&
GET(basic_variant<Allocator, Types...>&& v); // exposition only
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>&
GET(const basic_variant<Allocator, Types...>& v); // exposition only
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>&&
GET(const basic_variant<Allocator, Types...>&& v); // exposition only
```

Mandates: `I < sizeof...(Types)`.
Preconditions: `v.index()` is `I`.
Returns: A reference to the object stored in the variant `v`.

```
template<size_t I, class... Types>
```



```
constexpr variant_alternative_t<I, variant<Types...>>& get(variant<Types...>& v);
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&& get(variant<Types...>&& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&& get(const variant<Types...>&& v);
template<size_t I, class Allocator, class... Types>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>& get(basic_variant<Allocator, Types...>& v);
template<size_t I, class Allocator, class... Types>
constexpr variant_alternative_t<I, basic_variant<Allocator, Types...>>&& get(basic_variant<Allocator, Types...>&& v);
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>& get(const basic_variant<Allocator, Types...>& v);
template<size_t I, class Allocator, class... Types>
constexpr const variant_alternative_t<I, basic_variant<Allocator, Types...>>&& get(const basic_variant<Allocator, Types...>&& v);
```

Mandates: I < sizeof...(Types).

Effects: If v.index() is I, returns a reference to the object stored in the **variant** v. Otherwise, throws an exception of type bad_variant_access.

```
template<class T, class... Types> constexpr T& get(variant<Types...>& v);
template<class T, class... Types> constexpr T&& get(variant<Types...>&& v);
template<class T, class... Types> constexpr const T& get(const variant<Types...>& v);
template<class T, class... Types> constexpr const T&& get(const variant<Types...>&& v);
template<class T, class Allocator, class... Types> constexpr T& get(basic_variant<Allocator, Types...>& v);
template<class T, class Allocator, class... Types> constexpr T&& get(basic_variant<Allocator, Types...>&& v);
template<class T, class Allocator, class... Types> constexpr const T& get(const basic_variant<Allocator, Types...>& v);
template<class T, class Allocator, class... Types> constexpr const T&& get(const basic_variant<Allocator, Types...>&& v);
```

Mandates: The type T occurs exactly once in Types.

Effects: If v holds a value of type T, returns a reference to that value. Otherwise, throws an exception of type bad_variant_access.

```
template<size_t I, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>>
get_if(variant<Types...>* v) noexcept;
template<size_t I, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>>
get_if(const variant<Types...>* v) noexcept;
template<size_t I, class Allocator, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, basic_variant<Allocator, Types...>>>
get_if(basic_variant<Allocator, Types...>* v) noexcept;
template<size_t I, class Allocator, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, basic_variant<Allocator, Types...>>>
get_if(const basic_variant<Allocator, Types...>* v) noexcept;
```

Mandates: I < sizeof...(Types).

Returns: A pointer to the value stored in **the variant** v, if v != nullptr and v->index() == I. Otherwise, returns nullptr.

```
template<class T, class... Types>
constexpr add_pointer_t<T>
get_if(variant<Types...>* v) noexcept;
template<class T, class... Types>
constexpr add_pointer_t<const T>
get_if(const variant<Types...>* v) noexcept;
template<class T, class Allocator, class... Types>
constexpr add_pointer_t<T>
get_if(basic_variant<Allocator, Types...>* v) noexcept;
template<class T, class Allocator, class... Types>
constexpr add_pointer_t<const T>
get_if(const basic_variant<Allocator, Types...>* v) noexcept;
```

Mandates: The type T occurs exactly once in Types.

Effects: Equivalent to: return get_if<i>(v); with i being the zero-based index of T in Types.

Modify 22.6.6 Relational operators

```
template<class... Types>
constexpr bool operator==(const variant<Types...>& v, const variant<Types...>& w);
```

```

template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator==(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) == GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If v.index() != w.index(), false; otherwise if v.valueless_by_exception(), true; otherwise GET<i>(v) == GET<i>(w) with i being v.index().

template<class... Types>
constexpr bool operator!=(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator!=(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) != GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If v.index() != w.index(), true; otherwise if v.valueless_by_exception(), false; otherwise GET<i>(v) != GET<i>(w) with i being v.index().

template<class... Types>
constexpr bool operator<(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator<(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) < GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If v.valueless_by_exception(), false; otherwise if v.valueless_by_exception(), true; otherwise, if v.index() < w.index(), true; otherwise if v.index() > w.index(), false; otherwise GET<i>(v) < GET<i>(w) with i being v.index().

template<class... Types>
constexpr bool operator>(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator>(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) > GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If v.valueless_by_exception(), false; otherwise if w.valueless_by_exception(), true; otherwise, if v.index() > w.index(), true; otherwise if v.index() < w.index(), false; otherwise GET<i>(v) > GET<i>(w) with i being v.index().

template<class... Types>
constexpr bool operator<=(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator<=(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) <= GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If v.valueless_by_exception(), true; otherwise if w.valueless_by_exception(), false; otherwise, if v.index() < w.index(), true; otherwise if v.index() > w.index(), false; otherwise GET<i>(v) <= GET<i>(w) with i being v.index().

template<class... Types>
constexpr bool operator>=(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types>
constexpr bool operator>=(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Mandates: GET<i>(v) >= GET<i>(w) is a valid expression that is convertible to bool, for all i.
Returns: If w.valueless_by_exception(), true; otherwise if v.valueless_by_exception(), false; otherwise, if v.index() > w.index(), true; otherwise if v.index() < w.index(), false; otherwise GET<i>(v) >= GET<i>(w) with i being v.index().

template<class... Types> requires (three_way_comparable<Types> && ...)
constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
operator<=>(const variant<Types...>& v, const variant<Types...>& w);
template<class AllocatorV, class AllocatorW, class... Types> requires (three_way_comparable<Types> && ...)
constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
operator<=>(const basic_variant<AllocatorV, Types...>& v, const basic_variant<AllocatorW, Types...>& w);
Effects: Equivalent to:
    if (v.valueless_by_exception() && w.valueless_by_exception())
        return strong_ordering::equal;
    if (v.valueless_by_exception()) return strong_ordering::less;
    if (w.valueless_by_exception()) return strong_ordering::greater;
    if (auto c = v.index() <=> w.index(); c != 0) return c;
    return \exposid{GET}<i>(v) <=> \exposid{GET}<i>(w);
    with i being v.index().

```

Modify 22.6.7 Visitation

Let *as-variant* denote the following exposition-only function templates:

```

template<class... Ts>
    auto&& as-variant(variant<Ts...>& var) { return var; }
template<class... Ts>
    auto&& as-variant(const variant<Ts...>& var) { return var; }
template<class... Ts>
    auto&& as-variant(variant<Ts...>&& var) { return std::move(var); }
template<class... Ts>
    auto&& as-variant(const variant<Ts...>&& var) { return std::move(var); }
template<class Allocator, class... Ts>
    auto&& as-variant(basic_variant<Allocator, Ts...>& var) { return var; }
template<class Allocator, class... Ts>
    auto&& as-variant(const basic_variant<Allocator, Ts...>& var) { return var; }
template<class Allocator, class... Ts>
    auto&& as-variant(basic_variant<Allocator, Ts...>&& var) { return std::move(var); }
template<class Allocator, class... Ts>
    auto&& as-variant(const basic_variant<Allocator, Ts...>&& var) { return std::move(var); }

```

Let n be `sizeof...(Variants)`...

Modify [22.6.10](#) Specialized algorithms

```

template<class... Types>
    constexpr void swap(variant<Types...>& v, variant<Types...>& w) noexcept(see below);
template<class Allocator, class... Types>
    constexpr void swap(basic_variant<Allocator, Types...>& v, basic_variant<Allocator, Types...>& w) noexcept(see below);
    Constraints: is_move_constructible_v<Ti> && is_swappable_v<Ti> is true for all  $i$ .
    Effects: Equivalent to v.swap(w).
    Remarks: The exception specification is equivalent to noexcept(v.swap(w)).

```

Modify [22.6.12](#) Hash support

```

template<class Allocator, class... Types> struct hash<basic_variant<Allocator, Types...>>;
The specialization hash<basic_variant<Allocator, Types...>> is enabled (\[unord.hash\]) if and only if every specialization in hash<remove_const_t<Types...>> is enabled. The member functions are not guaranteed to be noexcept.

```